

LLM Selection Assistant

Maxim Tseitlin

The essence of the project

Motivation:

A tool for cost optimization

Use-case:

Let somebody writes program.

This program, particularly, sends prompts to private closed LLM models - so need to pay per token. He selected some prompts and decided to try other LLMs usage for them to decrease cost. - - -

For each of selected prompts he may use "LLM Selection Assistant" which:

- * will launch this prompt on relevant models from different LLM providers.
- * report launching results.

It will help to select the optimal LLM model for specified prompts launching.

Access to the project:

The project will have Web UI and will deployed on "Streamlit Community Cloud".

The project scope

What is optimized:

- Specified prompt

Modes:

- "English only"|"Multi-language"

Support for:

- tools calling

Common workflow & UI structure

Common workflow:

1. Select LLM providers for selected prompt testing
2. Prompt data preparation
3. Select judge LLM model for verification
4. For each LMM provider get models list which is appropriate for prompt launching
5. Lunch prompts set on each model in provider model lists. Keep answer and model statistics for (cost, in/out tokens number, response time)

UI Structure:

UI is implemented as wizard a wizard that guides the user through 3 steps:

- Select LLM providers
- Upload Prompt Evaluation Set
- Evaluate the Prompt Set

“Select LLM providers” Page

Provider Name	API KEY
Open AI	
GEMINI	GEMINI_API_KEY
ANTROPHIC	
...	
...	
...	

NEXT

Status line

"Provider Name" field is read-only

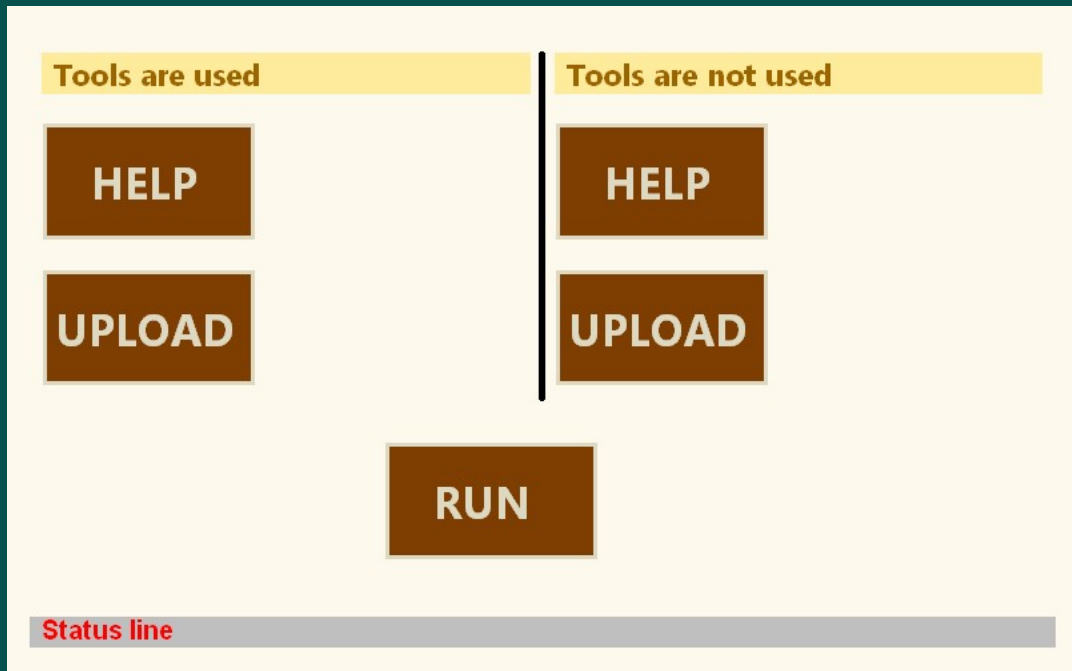
- Contains **supported** providers names

Pressing on [NEXT] button Validate API KEYS:

- On invalid key - error message is displayed
- If none API KEYS were entered - error message is displayed

If validation succeeded – next **"Upload Prompt Evaluation Set"** page will be displayed.

Upload Prompt Evaluation Set



Tools support needs additional data. That's why it's singled out as a separate case.

For both cases: with and without tools

- **"HELP" button** will give instructions how to gather data for evaluation
- Press on **"UPLOAD" button** will upload and validate uploaded set.

Errors will be displayed on "Status line".

If uploaded data was successfully validated, it will enable **"RUN" button**.

Pressing on [RUN] button will start evaluation.

The information about evaluation progress will be displayed on "Status line".

On evaluation finish, "Evaluation" page will be displayed.

“Evaluation” Page

Model	Avg Response time	Avg prompt cost
☒ gemini-2.5-flash	0.2	0.002\$
☐ gemini-2.5-pro	0.3	0.002\$
☐ gemini-2.0-flash	0.3	0.003\$
☐ gemma-3-1b-it	0.3	0.004\$
☐ gemini-3-pro-preview	0.3	0.005\$
☐ ...	...	...
☐ ...	...	...
☐ ...	...	...
☐		
☐		

SHOW ACCURACY

INCLUDE CoT

DOWNLOAD

NEW EVALUATION

Model: gemini-2.5-flash, Accuracy: 0.9

“Evaluation” Page - details

1. Uploaded prompts are launched against models of selected providers. Time & Cost statistics are gathered and are presented to user. On this stage, **answer correctness is not evaluated** to save time. Correctness evaluation is performed on selected model **by request only**.
2. Answer correctness is defined as correspondence to answer which was got on user side.
[SHOW ACCURACY] button displays accuracy on the selected model on status bar.
3. [DOWNLOAD] button zip and downloads all stuff about the selected models which was gathered till now.
4. Let the user is satisfied with the selected model statistics. The user may want to be convinced that this model makes reasoning correctly when answering the question. Then he may press [INCLUDE CoT] button. The Judge LLM model will check if the selected model supports CoT. (If not, then error message will be displayed on status line.) If CoT is supported, then the selected model will answer each question from "Prompts Set" with CoT & tools call dumping. The answers will be saved for further downloading.
5. Pressing [NEW EVALUATION] button will return to "Providers" page to start new evaluation.

Data preparation details

1. User saves full prompt for different parameters set (~ 20 prompts)
system_prompt.txt,
user_prompt01.txt, user_prompt02.txt, ...
answer01.txt, answer02.txt, ...
2. If tools may be called for this prompt:
 - * dump tools schema tools_schema.txt (A function to do this will be provided)
 - * for each prompt: save answer with tools calls dump for it:
tools_dump01.txt, tools_dump02.txt (prompt to do it will be provided)(If your model does not support this functionality - use state-of-the-art model of your LLM Provider for it)
3. Pack to .zip file
4. Upload .zip file via WebUI

Prompt for tools calls dumping

System prompt prefix:

Answer the PROMPT adhering to the following rules:

GOAL: To gather information required for PROMPT debugging.

Instructions for Tool Usage:

Whenever you call a function or tool, you must follow this structured output format:

- * Function Execution: Perform the necessary tool calls.
- * Fact Repository: Create a section titled `### TOOL_FACTS`.
List every piece of raw data returned by the tools as discrete, numbered facts.
For each entry, you must include:
 - Tool Name: The name of the function called.
 - Parameters: The exact arguments/parameters passed to the tool.
 - Raw Output: The return from the tool in JSON format.

Reasoning & Synthesis:

Create a section titled `### REASONING`.

- * You must use the facts listed in the `TOOL_FACTS` section to derive your conclusion.
- * Explain your reasoning step by step, then give the Final Answer.

Final Answer: Provide the concise response to the user.

Strict Rule: Do not internalize tool outputs into your prose until they have been "dumped" into the `TOOL_FACTS` section first.

PROMPT:

Tools calls dump example

PROMPT: What is the weather in Paris?

TOOL_FACTS

1. **Tool Name:** `google_search`
2. **Parameters:** `{"queries": ["current weather in Paris May 14 2026"]}`
3. **Raw Output:** `{"current_conditions": {"current_weather_conditions": {"temperature": "11°C", "wind": {"speed": "6 mph", "from_direction": "northwest"}, "humidity": "68%", "uv_index": "0", "precipitation": {"chance": "30%", "type": "rain"}, "part_of_day": "daytime", "condition": "light rain"}, "source_url": "[https://support.google.com/websearch/answer/13687874] (https://support.google.com/websearch/answer/13687874)", "source_title": "Google Weather", "corroboration_text": "This response uses data provided by Google Weather", "current_conditions_update_time": "2026-05-14T19:59:00+02:00", "forecast_location": "Paris, France"}}`

REASONING

The data from the `google_search` tool (Fact 1) provides the current weather conditions for Paris, France as of May 14, 2026. The temperature is recorded at 11°C with a condition of light rain. The wind is moving at 6 mph from the northwest, and the humidity is 68%.

Final Answer

The weather in Paris is currently 11°C with light rain. There is 68% humidity and a 6 mph wind coming from the northwest.

Judge LLM model

Select Judge LLM model as:

- * The state-of-the-art model of first supported provider in Providers list

Judge LLM model usage:

1. Models for prompt launching selection:
 - * get models list for specified provider.
 - * ask Judge LLM model to report which models from the list are relevant for the prompt answering
(We will test only these models.)
 2. Verify answer from selected model:
 - * We send prompts from user to selected model and we get responses.
 - * For each prompt we ask Judge LLM:
 - If the response is answer on the prompt correctly?
if yes, then we ask Judge LLM:
 - If the response contradicts the answer on this prompt from user data?
- Afterwards we calculate and display **model Accuracy**, basing on these tests results.

Tools preparation for evaluation run

If **tools** are used for **prompt resolution**, then we should to **build tools list** for our simulation basing on user site data.

Parameters set for each tool is described by JSON schema which is attached to user site data (**tools_schema.txt**).

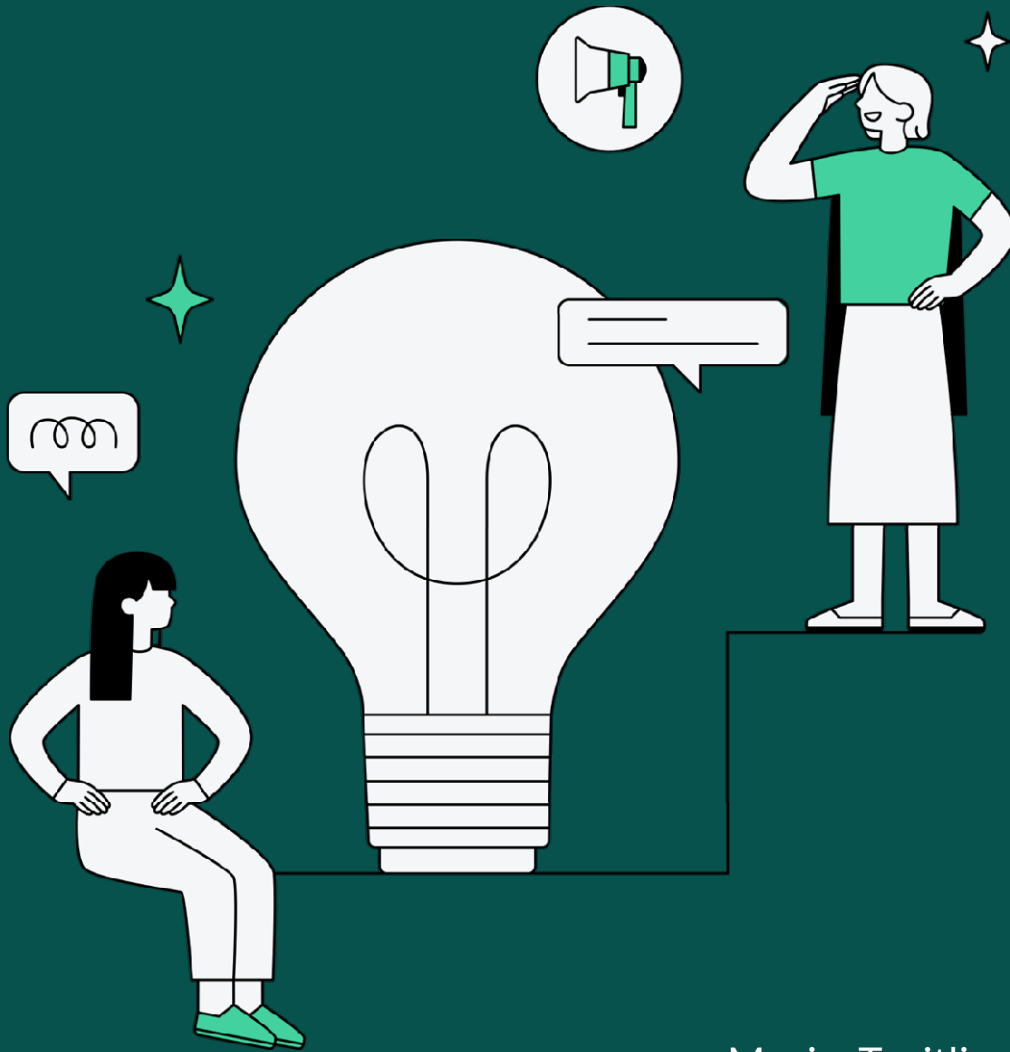
Calls to tool are logged in tools_dump files gathered on user side.

Return object is consistent and may be:

- * simple types (numeric, string, boolean)
- * list
- * dictionary

Based on this data, we will create python file, containing stubs for functions and tools list and import it.

(if tools dump contains several calls for one tool - stub will return these values one by one)



THANK
YOU

Maxim Tseitlin

Credits

This presentation template is free for
everyone to use thanks to the following:



for the presentation template

Pexels & Pixabay

for the photos